

TCC805x MCU BSP

API Specification for Pulse Density Modulation

Rev. 1.00 [A]

2020-11-12

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files	4
4	Constants.....	5
4.1	Connection Type	5
4.2	Channel Information Values.....	6
4.3	PDM Mode Value	7
5	Structures	8
5.1	PDMModeConfig_t	8
5.2	PDMChannelHandler_t	10
6	APIs	11
6.1	PDM_EnableClock	11
6.2	PDM_DisableClock	11
6.3	PDM_Init	12
6.4	PDM_Deinit	12
6.5	PDM_GetChannelStauts.....	13
6.6	PDM_Enable.....	14
6.7	PDM_Disable	15
6.8	PDM_SetConfig.....	16
7	How to use PDM Driver	17
7.1	Operation of PDM	17
7.2	Example of Using PDM Driver	17
8	References	18
9	Revision History	19
	Rev. 1.00: 2020-11-12	19

Figures

Figure 3.1	Location of Source Files	4
Figure 7.1	Block Diagram of PDM Driver.....	17

Tables

Table 2.1	Terms and Abbreviation	3
-----------	------------------------------	---

1 INTRODUCTION

This document describes the pulse density modulation device driver (hereinafter referred to as PDM driver) of TCC805x MCU BSP. For more detailed information of the pulse density modulation in the TCC805x MCU Sub-system, refer to Chapter 14 Pulse Density Modulation (PDM) of “*PART 11-MICOM SUB-SYSTEM*” in the “*TCC805x Full Specification*”. [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

PDM	Pulse Density Modulation
PMM	Pulse Monitoring Module

3 SOURCE FILES

3.1 Location of Source Files

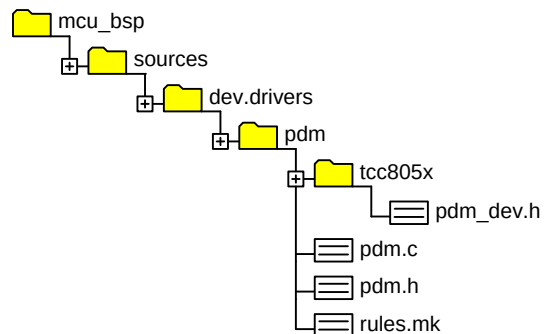


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- pdm_dev.h
 - PDM specific header according to chipset
- pdm.c
 - PDM common source code
- pdm.h
 - PDM common header includes API & DEFINE for user.
- rules.mk
 - Make rules

4 CONSTANTS

4.1 Connection Type

Constant values for the PDM output connection type to the PMM. Default value of the PDM connection type is EXTERNAL. The PDM connection type is used only to check safety mechanism.

Declaration

```
#define PDM_CONNECT_INTERNAL (0x00000000UL)
#define PDM_CONNECT_EXTERNAL (0x00000001UL)
```

Description

Value	Description
PDM_CONNECT_INTERNAL	PDM monitoring module checks the PDM output that is internally connected. (internal loopback)
PDM_CONNECT_EXTERNAL	PDM monitoring module checks the PDM output that is externally connected. (external loopback)

Remark

None

4.2 Channel Information Values

Constant values for the operating information of PDM channel

Declaration

```
#define PDM_SAFETY_FEATURE_ENABLED

#define PDM_STATE_BUSY          (1UL)
#define PDM_STATE_IDLE         (0UL)

#define PDM_ON                  (1UL)
#define PDM_OFF                 (0UL)

#ifdef PDM_SAFETY_FEATURE_ENABLED
#define PMM_ON                  (1UL)
#define PMM_OFF                 (0UL)
#endif

/* The PWM controller consist of 3 PWM modules (PWM-0/1/2). */
#define PDM_TOTAL_MODULES      (3UL)

/* Each PWM module has four separate PWM blocks (PWM-A/B/C/D). */
#define PDM_TOTAL_CH_PER_MODULE (4UL)

/* total 12channels */
#define PDM_TOTAL_CHANNELS      PDM_TOTAL_MODULES * PDM_TOTAL_CH_PER_MODULE
```

Description

Value	Description
<i>PDM_SAFETY_FEATURE_ENABLED</i>	<i>Define safety feature</i>
<i>PDM_STATE_BUSY</i>	<i>Busy state of PDM</i>
<i>PDM_STATE_IDLE</i>	<i>Idle state of PDM</i>
<i>PDM_ON</i>	<i>PDM is enabled.</i>
<i>PDM_OFF</i>	<i>PDM is disabled.</i>
<i>PMM_ON</i>	<i>PMM is enabled.</i>
<i>PMM_OFF</i>	<i>PMM is disabled.</i>
<i>PDM_TOTAL_MODULES</i>	<i>The number of PDM modules</i>
<i>PDM_TOTAL_CH_PER_MODULE</i>	<i>The number of PDM channels per module</i>
<i>PDM_TOTAL_CHANNELS</i>	<i>The number of total PDM channels</i>

Remark

None

4.3 PDM Mode Value

Constant values for the PDM mode

Declaration

```
/* PDM MAX clk 400Mhz */
#define PDM_PERI_CLOCK          400 * 1000 * 1000

/* Clock divide max value 3(divide 16) */
#define PDM_DIVID_MAX          3

/*Operation Mode Phase 1 & 2*/
#define PDM_OUTPUT_MODE_PHASE_1    0b0001UL
#define PDM_OUTPUT_MODE_PHASE_2    0b1001UL

/*Operation Mode Register 1 & 2*/
#define PDM_OUTPUT_MODE_REGISTER_1  0b0010UL
#define PDM_OUTPUT_MODE_REGISTER_2  0b0100UL
#define PDM_OUTPUT_MODE_REGISTER_3  0b0110UL

#define REG_OUTPUT_PATTERN_HIGH      0xFFFFFFFFUL
#define REG_OUTPUT_PATTERN_LOW       0x00000000UL
```

Description

Value	Description
PDM_PERI_CLOCK	PDM peripheral clock value
PDM_DIVID_MAX	Maximum clock divide value, divide by 16
PDM_OUTPUT_MODE_PHASE_1	Phase mode 1
PDM_OUTPUT_MODE_PHASE_2	Phase mode 2
PDM_OUTPUT_MODE_REGISTER_1	Register mode 1
PDM_OUTPUT_MODE_REGISTER_2	Register mode 2
PDM_OUTPUT_MODE_REGISTER_3	Register mode 3
REG_OUTPUT_PATTERN_HIGH	High output pattern of register mode
REG_OUTPUT_PATTERN_LOW	Low output pattern of register mode

Remark

Refer to the description for each output mode in “*TCC805x Full Specification*”. [1]

5 STRUCTURES

5.1 PDMModeConfig_t

Structure to store the PDM output mode information of each channel

Declaration

```
typedef struct _PDMModeConfig_  
{  
    uint32 mcPortNumber;  
    uint32 mcOperationMode;  
    uint32 mcClockDivide;  
    uint32 mcLoopCount;  
    uint32 mcInversedSignal;  
    uint32 mcPeriodNanoSec1;  
    uint32 mcDutyNanoSec1;  
    uint32 mcPeriodNanoSec2;  
    uint32 mcDutyNanoSec2;  
    uint32 mcPosition1;  
    uint32 mcPosition2;  
    uint32 mcPosition3;  
    uint32 mcPosition4;  
    uint32 mcOutPattern1;  
    uint32 mcOutPattern2;  
    uint32 mcOutPattern3;  
    uint32 mcOutPattern4;  
    uint32 mcMaxCount;  
} PDMModeConfig_t;
```


Description

Value	Description
<i>mcPortNumber</i>	<i>PDM output port number. Refer to “TCC805x Chip Specification”.</i>
<i>mcOperationMode</i>	<i>PDM operation Mode, Phase Mode 1, 2 and Register Mode 1, 2, 3</i>
<i>mcClockDivide</i>	<i>Clock divide value 0: Divide by 2 1: Divide by 4 2: Divide by 8 3: Divide by 16</i>
<i>mcLoopCount</i>	<i>Period cycle count. If this value is 0, PDM operates infinite loop mode.</i>
<i>mcInversedSignal</i>	<i>Inversed output signal</i>
<i>mcPeriodNanoSec1</i>	<i>Period1 value in Nano second scale for phase mode</i>
<i>mcDutyNanoSec1</i>	<i>Duty1 value in Nano second scale for phase mode</i>
<i>mcPeriodNanoSec2</i>	<i>Period2 value in Nano second scale for phase mode</i>
<i>mcDutyNanoSec2</i>	<i>Duty2 value in Nano second scale for phase mode</i>
<i>mcPosition1</i>	<i>Register PSTN1 value for Phase mode 1 and 2</i>
<i>mcPosition2</i>	<i>Register PSTN2 value for Phase mode 1 and 2</i>
<i>mcPosition3</i>	<i>Register PSTN1 value for Phase mode 2</i>
<i>mcPosition4</i>	<i>Register PSTN1 value for Phase mode 2</i>
<i>mcOutPattern1</i>	<i>Register OUT_PTN1 value for Register mode 1, 2, and 3</i>
<i>mcOutPattern2</i>	<i>Register OUT_PTN2 value for Register mode 2 and 3</i>
<i>mcOutPattern3</i>	<i>Register OUT_PTN3 value for Register mode 2 and 3</i>
<i>mcOutPattern4</i>	<i>Register OUT_PTN4 value for Register mode 2 and 3</i>
<i>mcMaxCount</i>	<i>Register MAX_CNT value for Register mode 3. The count range is from 1 to 127.</i>

Remark

None

5.2 PDMChannelHandler_t

Structure to store the PDM channel handle information about each channel

Declaration

```
typedef struct _PDMChannelHandler_
{
    uint32          chModuleId;
    uint32          chChannelId;
    uint32          chEnable;
    uint32          chOutSignalInIdle;
    uint32          chIdleState;
    PDMModeConfig_t chModeCfgInfo;

#ifdef PDM_SAFETY_FEATURE_ENABLED
    uint32          chPMMEnable;
    uint32          chPMMInputCnect;
    uint32          chPMMFaultStatus;
    uint32          chPMMTimeoutValue;
    uint32          chPMMDutyMargin;
#endif
} PDMChannelHandler_t;
```

Description

Value	Description
<i>chModuleId</i>	<i>PDM Module ID PDM0/1/2</i>
<i>chChannelId</i>	<i>PDM Channel ID 0/1/2/3</i>
<i>chEnable</i>	<i>PDM Channel Status, PDM_ON or PDM_OFF</i>
<i>chOutSignalInIdle</i>	<i>PDM output signal in idle mode</i>
<i>chIdleState</i>	<i>Idle state of PDM</i>
<i>chModeCfgInfo</i>	<i>Configuration value of PDM operation mode</i>
<i>chPMMEnable</i>	<i>PMM Monitoring Status, PMM_ON or PMM_OFF.</i>
<i>chPMMInputCnect</i>	<i>Input connection type of monitoring module, Internal or External connection</i>
<i>chPMMFaultStatus</i>	<i>Fault status detected by PMM</i>
<i>chPMMTimeoutValue</i>	<i>PMM timeout value</i>
<i>chPMMDutyMargin</i>	<i>PMM duty margin value</i>
<i>chPMMErrChannel</i>	<i>PMM error channel. If this value is TRUE, it means the PDM channel fault occurs.</i>

Remark

None

6 APIs

6.1 PDM_EnableClock

Function to enable peripheral clock for all PDM modules. Default clock rate is 400 MHz (= Maximum rate).

Declaration

```
void PDM_EnableClock  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

6.2 PDM_DisableClock

Function to disable peripheral clock for all PDM modules.

Declaration

```
void PDM_DisableClock  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

6.3 PDM_Init

Function to initialize all PDM channels. If this function is called, [PDMChannelHandler](#) structure for all PDM channels is filled with default value. And the PDM peri clock rate for each module is set and the PDM peri clock is enabled. If the PDM safety feature is enabled, this function registers FMU interrupt handler and enables FMU interrupts.

Declaration

```
sint32 PDM_Init  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

6.4 PDM_Deinit

Function to de-initialize all PDM channels. If this function is called, [PDMChannelHandler](#) structure for all channels is filled with 0.

Declaration

```
void PDM_Deinit  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

6.5 PDM_GetChannelStauts

Function to get the PDM channel status (BUSY or IDLE). When the PDM channel is enabled or the PDM output mode is set, this function is called.

Declaration

```
uint32 PDM_GetChannelStauts  
(  
    uint32 uiChannel  
);
```

Parameters

Value	Description
<i>uiChannel</i>	<i>PDM Channel value (0 ~ 11)</i>

Return

Value	Description
<i>PDM_STATE_BUSY</i>	<i>The PDM is in Busy state. (= 0)</i>
<i>PDM_STATE_IDLE</i>	<i>The PDM is in IDLE state. (= 1)</i>

Remark

None

6.6 PDM_Enable

Function to start generating the PDM signal.

For more information, refer to “*TCC805x Full Specification*”. (PDM Enable Register (OP_EN)) [1]

PDM EN sequence is as follows: Set EN → Set VUP → Set TRIG.

Declaration

```
SALRetCode_t PDM_Enable  
(  
    uint32 uiChannel,  
    uint32 uiMonitoring  
);
```

Parameters

Value	Description
<i>uiChannel</i>	<i>PDM Channel value 0 ~ 11</i>
<i>uiMonitoring</i>	<i>Enable the PMM monitoring module for safety feature.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.7 PDM_Disable

Function to stop generating the PDM output signal.

Declaration

```
SALRetCode_t PDM_Disable  
(  
    uint32 uiChannel,  
    uint32 uiMonitoring  
);
```

Parameters

Value	Description
<i>uiChannel</i>	<i>PDM Channel value 0 ~ 11</i>
<i>uiMonitoring</i>	<i>Disable the PMM monitoring module for safety feature.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.8 PDM_SetConfig

Function to set configuration values of the PDM registers. Main configuration is the PDM operation mode.

Declaration

```
SALRetCode_t PDM_SetConfig
(
    uint32          uiChannel,
    PDMModeConfig_t * pModeConfig
);
```

Parameters

Value	Description
<i>uiChannel</i>	<i>PDM Channel value (0 ~ 11)</i>
<i>pModeConfig</i>	<i>Pointer of configuration structure. Refer to the PDMModeConfig_t.</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

7 HOW TO USE PDM DRIVER

The PDM driver can generate pulse signal according to user configuration such as the PDM mode, duty, period, etc. To get more information about signal configuration of the PDM mode, refer to the description of PDM signal in “*TCC805x Full Specification*”.

[1]

7.1 Operation of PDM

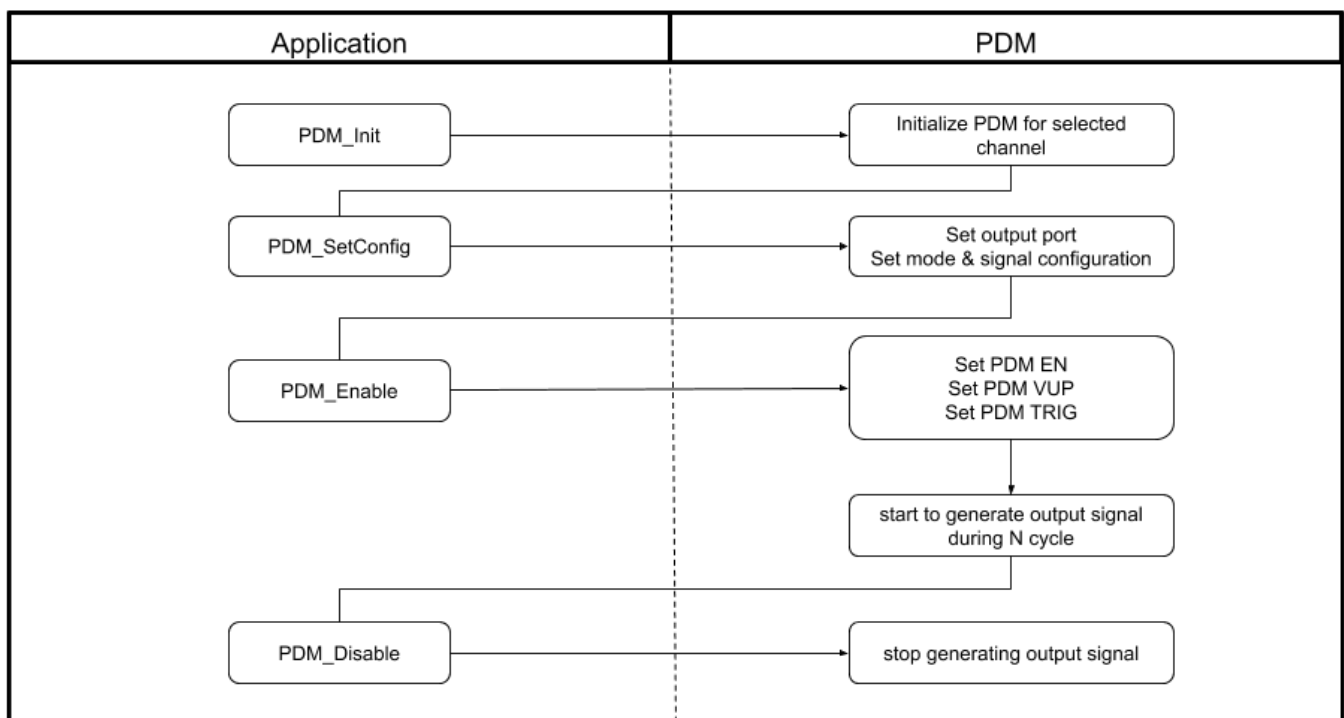


Figure 7.1 Block Diagram of PDM Driver

7.2 Example of Using PDM Driver

```

{
    // channel initializes
    PDM_Init();

    // set configuration. (port, mode, signal)
    PDM_SetConfig();

    // start to generate
    PDM_Enable();
}

```

8 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

9 REVISION HISTORY

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."